produces:

```
{:firstname=>"Fred", :lastname=>"Flintstone"}
```

Ruby will infer that the value should come from a variable with the same name as the key. If you try to use a key shortcut and there is no such local variable, Ruby will throw an error.

Compared with arrays, hashes have one significant advantage: they can use any object as an index. And you'll find something that might be surprising: Ruby remembers the order in which you add items to a hash. When you subsequently iterate over the entries, Ruby will return them in that order.

You'll find that hashes are one of the most commonly used data structures in Ruby. Later, Chapter 5, More About Methods, on page 85 lists more of the methods implemented by class Hash.

Digging

Often data is not simply a single hash or array, but comes in a complex package that combines hashes and arrays. Accessing data in a complicated structure can be a pain, but Ruby provides a shortcut with the dig method.

The dig method, which is defined for Array, Hash, and Struct, allows you to "dig" through a complicated data structure in a single command.

```
data = {
  mcu: [
    {name: "Iron Man", year: 2010, actors: ["Robert Downey Jr.", "Gwyneth Paltrow"]}
  ],
  starwars: [
    {name: "A New Hope", year: 1977, actors: ["Mark Hamill", "Carrie Fisher"]}
  ]
}
data[:mcu][0][:actors][1]     # => "Gwyneth Paltrow"
data.dig(:mcu, 0, :actors, 1) # => "Gwyneth Paltrow"
```

The biggest advantage of using dig is that if an element is not in the data structure, you return nil and not an exception.

Word Frequency: Using Hashes and Arrays

Let's round off this discussion of hashes and arrays with a simple program that calculates the number of times each word occurs in some text. (So, for example, in this sentence, the word *the* occurs two times.)

The problem breaks down into two parts. First, given some text as a string, return a list of words. That sounds like an array. Then, build a count for each distinct word. That sounds like a use for a hash—we can index it with the word and use the corresponding entry to keep a count.

Let's start with the method that splits a string into words:

tut_containers/word_freq/words_from_string.rb

```
def words_from_string(string)
  string.downcase.scan(/[\w']+/)
end
```